



Levius

DOCUMENTO TÉCNICO

Multitenant com RLS + servidor MCP

Uma fundação segura para produtos SaaS em que vários clientes compartilham o mesmo banco e uma IA consulta esses dados.

Projeto	Prova de conceito · prontosarios-poc
Tipo	Documento técnico de arquitetura
Data	08/10/2026
Versão	v1.0
Classificação	Confidencial

Resumo executivo

Dá para isolar clientes num banco compartilhado sem depender da disciplina de quem escreve código, e colocar uma IA em cima disso sem ampliar a superfície de risco. 43 casos de teste sustentam a afirmação, no banco e através do servidor MCP.

43

casos de teste passando:
23 no banco, 20 via
servidor MCP

4

ferramentas de domínio
no servidor MCP, e nada
além delas

0

chaves que ignoram a
RLS no caminho da
consulta

~30 min

para reproduzir tudo
num Supabase gratuito

Duas clínicas fictícias usam o mesmo Postgres. O médico de uma nunca alcança o paciente da outra, e quem garante isso é o banco, por Row Level Security, não o código da aplicação. Por cima existe um servidor MCP, o protocolo pelo qual um assistente de IA usa ferramentas: cada ferramenta roda como o usuário logado, sob as mesmas regras.

O que falta para virar produto não é arquitetura. São três frentes: OAuth para clientes externos, CI completo e validação com dado e volume reais.

1	Contexto e objetivo	a pergunta que o projeto responde
2	Arquitetura	stack e caminho de uma consulta
3	Como o isolamento funciona	schema, policies e garantias
4	O servidor MCP	ferramentas e regras de segurança
5	Evidência	baterias de teste e controle negativo
6	Como reproduzir	passo a passo e comandos
7	Aplicabilidade e próximos passos	o que se aproveita e o que falta

1 Contexto e objetivo

O projeto responde a uma pergunta de arquitetura, não de produto: é seguro deixar uma IA consultar o banco de um sistema multitenant de saúde?

O risco

Dois movimentos comuns em SaaS se cruzam num ponto específico: atender várias organizações no mesmo banco e oferecer IA sobre os dados delas.

- Num sistema multitenant, o pior desfecho é o profissional de uma organização alcançar o prontuário de outra.
- O caminho mais comum para isso é alguém esquecer um filtro por organização em uma consulta.
- Com uma IA no circuito, o problema piora: o que ela pede não é previsível e pode ser influenciado pelo texto que ela lê.

A decisão testada

Tirar a responsabilidade do isolamento do código da aplicação e colocá-la no banco.

PRINCÍPIO

A regra de separação vive nas policies, avaliada a partir do `auth.uid()` do token. Uma consulta sem filtro não vaza nada, porque o filtro já aconteceu antes. A IA pede o que quiser; quem decide o que entregar é o Postgres.

Ordem do trabalho

1. **Etapa 1, multitenant com RLS.** Banco isolado, sem nenhum MCP, num schema mínimo com dois tenants reais e testes que provam o isolamento.
2. **Etapa 2, servidor MCP.** Uma Supabase Edge Function em que cada ferramenta roda como o usuário logado, portanto sob as policies da etapa 1.

A ordem foi deliberada. Depurar vazamento entre tenants com o protocolo e um modelo no meio do caminho é muito mais caro do que depurar policy direto no banco.

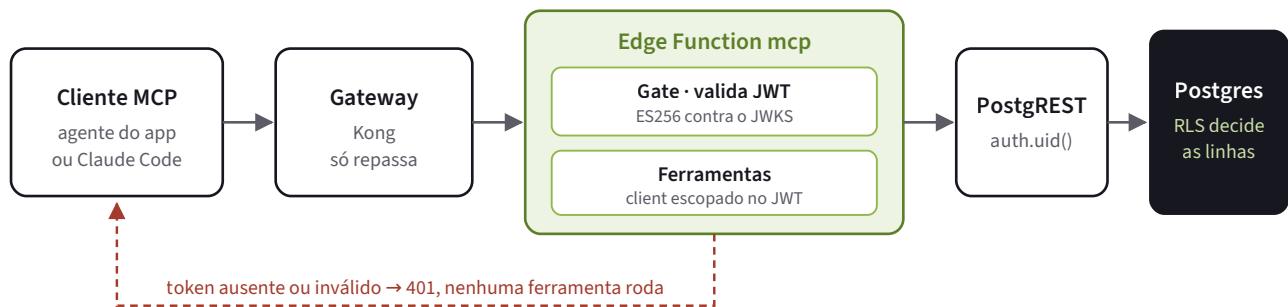
Escopo

Incluído	Fora do escopo
Schema mínimo com isolamento por RLS	OAuth e consentimento para clientes externos
Servidor MCP com 4 ferramentas de domínio	Interface de produto para usuário final
Duas baterias de teste, 43 casos	Deploy em produção e integração com app real
Tela de demonstração em HTML estático	Qualquer dado de saúde real

2 Arquitetura

Um repositório Node/TypeScript que opera um projeto Supabase. A consulta atravessa três camadas, e a de baixo, o banco, é a que manda.

Caminho de uma consulta



- 1 Usuário autentica no Supabase e recebe um JWT assinado com chave assimétrica.
- 2 O cliente envia o JWT no cabeçalho Authorization de uma chamada MCP.
- 3 A Edge Function valida o token contra o JWKS e monta um client Supabase escopado nele.
- 4 A ferramenta executa a consulta sem nenhum filtro de organização.
- 5 A RLS decide quais linhas existem para aquele usuário.

Stack

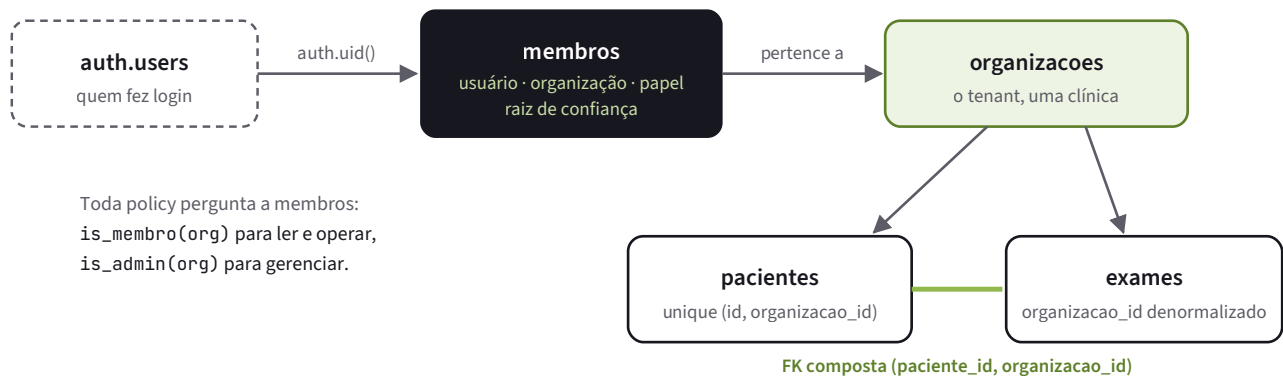
Peça	Papel no projeto
Supabase / Postgres	Tabelas, policies de RLS e autenticação dos usuários
Supabase Edge Functions	O servidor MCP, em Deno
@supabase/mcp	Block oficial, base do servidor, instalado e customizado
TypeScript + tsx	Scripts de seed, testes e utilitários
HTML estático	Tela de demonstração, sem framework e sem dependência

Por que `verify_jwt = false` no gateway

A própria função verifica o token com `withSupabase`, contra o JWKS. Assim ela devolve o `401` com o desafio `WWW-Authenticate` que clientes MCP esperam e aceita tanto tokens de sessão do app quanto, no futuro, tokens OAuth. Desligar a checagem no gateway não deixa a função aberta: quatro casos de teste provam isso.

3 Como o isolamento funciona

Toda a segurança se apoia numa única cadeia de pertencimento, com quatro tabelas. `membros` é a raiz de confiança: todas as outras policies perguntam a ela.



Cadeia de pertencimento. Exames pertencem a um paciente e, transitivamente, a uma organização.

Policies por tabela

Tabela	SELECT	INSERT	UPDATE	DELETE
<code>organizacoes</code>	<code>is_membro</code>	—	—	—
<code>membros</code>	<code>is_membro</code>	<code>is_admin</code>	<code>is_admin</code>	<code>is_admin</code>
<code>pacientes</code>	<code>is_membro</code>	<code>is_membro</code>	<code>is_membro (USING + WITH CHECK)</code>	<code>is_admin</code>
<code>exames</code>	<code>is_membro</code>	<code>is_membro</code>	<code>is_membro (USING + WITH CHECK)</code>	<code>is_admin</code>

Eixo de papel: o **médico opera** (lê, cria e edita pacientes e exames); o **admin gerencia** (além disso, deleta e mexe em `membros`). `membros` é a tabela mais fechada do modelo: se um médico pudesse inserir ali, se adicionaria à outra clínica e todo o isolamento cairia junto.

Garantias estruturais

A coluna `exames.organizacao_id` é denormalizada para a policy virar uma comparação direta com índice, em vez de um subselect em `pacientes` por linha. Para que ela não possa divergir, quem garante é o banco: `unique (id, organizacao_id)` em `pacientes` e chave estrangeira composta em `exames`.

EFEITO PRÁTICO

Um exame apontando para um paciente de outra organização é impossível de inserir, inclusive com `service_role`. É garantia estrutural, não convenção: sobrevive a pressa, a refactor e a permissão de administrador.

Por que os helpers são `SECURITY DEFINER`

A policy de SELECT de `membros` precisa consultar `membros`. Escrita inline, o Postgres reaplica a RLS dentro da subconsulta, de novo e de novo, até o erro `42P17 infinite recursion`. Os helpers `is_membro(uuid)` e `is_admin(uuid)` rodam com os privilégios do dono da tabela e quebram o ciclo, com três cuidados:

- `set search_path = ''` e nomes qualificados, para a resolução de nomes não ser sequestrada por uma tabela homônima;
- `stable`, para o planner avaliar uma vez em vez de uma vez por linha;
- nunca `force row level security` em `membros`: com FORCE, até o dono é filtrado e o helper deixa de enxergar.

USING vs WITH CHECK no UPDATE: o que foi medido

O ataque é o contrabando de tenant: atualizar a `organizacao_id` de um paciente próprio para a outra clínica.

```
update pacientes set organizacao_id = '<org B>' where id = '<paciente da org A>';
```

A intuição comum é que sem `WITH CHECK` isso passa. Medido no harness local, não é bem assim: o Postgres valida o valor novo em duas camadas.

1. O `WITH CHECK` da policy de UPDATE; se omitido, o próprio `USING` é reaproveitado como check.
2. O `USING` das policies de SELECT, porque não se pode atualizar uma linha para um estado em que não se conseguiria mais lê-la.

O vazamento só aconteceu com as duas camadas frouxas ao mesmo tempo. O `WITH CHECK` explícito fica mesmo assim, para a escrita não depender do efeito colateral de outra policy. Se amanhã a leitura for afrouxada, a proteção da escrita não vai junto.

Em `UPDATE`, os dois cumprem papéis diferentes: uma tentativa fora do escopo afeta **zero linhas** (USING), e mover um registro para outra organização é bloqueado com **erro explícito** (WITH CHECK).

RLS não substitui GRANT

São dois portões em série. Sem `GRANT`, o Postgres nem avalia a policy; sem RLS, o `GRANT` libera a tabela inteira. As migrations fazem os dois: revogam tudo de `anon` e concedem explicitamente a `authenticated`.

4 O servidor MCP

Uma Edge Function com quatro ferramentas de domínio. O servidor MCP oficial da Supabase roda com `service_role` e ignora a RLS; este faz o oposto: toda ferramenta roda como o usuário logado.

1 Gate

Só passa JWT de usuário com assinatura válida contra o JWKS. Falhou: 401.

2 Ferramenta

Input validado por Zod e regra de negócio antes de escrever.

3 Banco

RLS e FK composta da etapa 1. As ferramentas não filtram por organização; se uma tivesse bug, a policy ainda barraria. É a camada que manda.

Defesa em profundidade: três camadas em série.

Ferramentas

Ferramenta	Leitura	Devolve	Regra além da RLS
<code>whoami</code>	sim	id, e-mail, role e <code>client_id</code> do token verificado	<code>client_id</code> é nulo em sessão do app; vem preenchido em token OAuth
<code>listar_pacientes</code>	sim	pacientes e total	Nenhuma, de propósito: sem filtro de org no código, a RLS decide
<code>buscar_examenes</code>	sim	paciente e exames, do mais recente ao mais antigo	O paciente precisa ser visível ao usuário
<code>criar_exame</code>	não	o exame criado	Organização vem da linha do paciente lida do banco; autor vem do token; campo extra é rejeitado

As três primeiras são marcadas como somente leitura (`readOnlyHint`), e o cliente lê isso antes de chamar.

Regras de segurança

Regra	Como se cumpre
Só o client do usuário	As ferramentas recebem um client escopado pelo JWT. Não existe <code>service_role</code> em nenhum lugar de <code>supabase/functions/</code> .
Nada de SQL livre	Ferramentas de domínio com schema estrito, UUID tipado e limites de tamanho. Um agente não tem como pedir outra coisa.
Nada editável decide autorização	Organização vem do banco e autor vem do token verificado, nunca de <code>user_metadata</code> nem do input.
Erro sem vazamento lateral	"Paciente não existe" e "paciente é de outra organização" geram a mesma mensagem, para ninguém sondar ids de outros tenants.

Regra	Como se cumpre
Gate antes das ferramentas	Sem credencial válida, <code>401</code> antes de qualquer ferramenta. A chave pública do projeto não serve como credencial de usuário.
Token fica no backend	No modo embarcado, o access token fica no orquestrador do agente, nunca no prompt nem exposto ao provedor do modelo.

REQUISITO FÁCIL DE ESQUECER

O servidor valida o token contra chaves assimétricas (ES256 ou RS256) e recusa o formato simétrico antigo, HS256. Em projeto Supabase novo já é o padrão; em projeto antigo, migre em Project Settings → JWT Keys antes de usar.

5 Evidência

Duas baterias, 43 casos, todos passando, executadas no Supabase local e contra um projeto hospedado. "Não vi vazamento" não é prova de isolamento; estes testes são.

Bateria	Casos	O que cada caso faz
<code>npm run test:rls</code>	23	Login real e tentativa de operação proibida, pelo caminho anon key → login → JWT ES256 → PostgREST
<code>npm run test:mcp</code>	20	As mesmas provas atravessando o servidor MCP, incluindo quatro de credencial inválida

CONTROLE COM SERVICE_ROLE

As duas baterias leem as contagens reais com a chave que ignora a RLS (Alfa: 2 pacientes e 3 exames; Beta: 2 e 2). Sem esse controle, uma lista vazia provaria apenas ausência de dado, não isolamento.

Casos que valem citar numa conversa técnica

Caso	O que demonstra
Exame que cruza tenant barrado pela FK composta (<code>23503</code>), mesmo com <code>service_role</code>	Garantia estrutural acima de qualquer permissão
Médico insere exame no próprio paciente, mas o <code>delete</code> dele afeta 0 linhas	Eixo de papel funcionando por USING
Admin da Alfa não escreve em <code>membros</code> da Beta (<code>42501</code>)	Papel não atravessa tenant

Caso	O que demonstra
Organização injetada no input de <code>criar_exame</code> rejeitada antes de escrever	Nada editável decide autorização
Chave pública do projeto usada como Bearer recebe <code>401</code>	O gate distingue credencial de projeto de credencial de usuário
Consulta sem autenticar recebe <code>permission denied</code>	RLS e GRANT estão os dois no lugar

Saída da bateria MCP (trecho)

```

=== SEM CREDENCIAL VALIDA (o gate barra antes de qualquer tool) ===
OK    1. sem Authorization: 401
OK    4. anon key (credencial de projeto, nao de usuario) como Bearer: 401
=== MEDICO DA CLINICA ALFA, VIA MCP ===
OK    8. listar_pacientes: so os da Alfa
      → recebeu 2 de 4 pacientes do banco; 0 de outra org
OK   10. buscar_exames(paciente da Beta): erro, nenhum dado
OK   12. criar_exame com organizacao_id da Beta no input: rejeitado
      → Unrecognized key: "organizacao_id"
OK   14. criar_exame(paciente da Alfa) FUNCIONA: org vem do paciente
=====
TODOS OS 20 CASOS PASSARAM - isolamento entre tenants confirmado.

```

Controle negativo: a suíte sabe falhar

Uma suíte que só diz "OK" não vale nada se não souber falhar. A policy de leitura de `pacientes` foi trocada temporariamente para `using (true)` no banco local, e a bateria MCP rodou de novo.

```

FALHOU 8. listar_pacientes: so os da Alfa
      → recebeu 4 de 4 pacientes do banco; 2 de outra org
FALHOU 10. buscar_exames(paciente da Beta): erro, nenhum dado
OK    11. criar_exame(paciente da Beta): erro, Beta com os mesmos exames
      → [42501] new row violates row-level security policy for table "exames"
FALHOU 16. listar_pacientes: so os da Beta
FALHOU 18. buscar_exames(paciente da Alfa): erro, nenhum dado

```

- **O vazamento aparece pelo MCP.** O nome do paciente da Beta chega ao médico da Alfa, e os casos 8, 10, 16 e 18 pegam isso. Os exames continuam vazios porque a policy de `exames` ficou intacta: cada tabela se defende sozinha.
- **O caso 11 continua passando, e esse é o ponto.** A checagem de negócio da ferramenta deixou passar, mas a policy de INSERT de `exames` barrou a escrita. É a defesa em profundidade funcionando.

Na bateria do banco, dois controles negativos equivalentes: dar o DELETE de pacientes ao médico faz o caso 8 falhar; afrouxar leitura e escrita ao mesmo tempo faz o paciente efetivamente migrar de tenant.

Verificação manual no projeto hospedado

Impersonando cada usuário no SQL Editor com `select nome, organizacao_id from pacientes;`:

Rodando como	Linhas devolvidas
<code>medico.alfa@exemplo.test</code>	2: Ana Alves e Artur Braga (Alfa)
<code>medico.beta@exemplo.test</code>	2: Bruna Costa e Bento Dias (Beta)
<code>postgres</code> (dono da tabela, sem RLS)	4: as duas clínicas

A terceira linha dá sentido às outras duas: os quatro pacientes existem, então o "2" não é tabela vazia, é a policy filtrando.

6 Como reproduzir

Cerca de 30 minutos num projeto Supabase gratuito. Pré-requisitos: conta no Supabase e Node.js 22 ou mais novo; GitHub, Vercel e Docker são opcionais.

1. Instalar dependências com `npm install`.
2. Criar o projeto no Supabase e guardar a senha do banco, que não é mostrada de novo.
3. Trocar as chaves de JWT para assimétricas em Project Settings → JWT Keys. Obrigatório e fácil de esquecer.
4. Preencher o `.env` com `SUPABASE_URL`, `SUPABASE_ANON_KEY` e `SUPABASE_SERVICE_ROLE_KEY`.
5. Criar tabelas e regras pelo SQL Editor (`schema-completo.sql` e `seed.sql`) ou pela CLI (`npx supabase db push`).
6. Popular os dados com `npm run seed` : quatro usuários e as duas clínicas.
7. Primeira prova, `npm run test:rls` : 23 casos. Se algum falhar, pare: não adianta subir o MCP sobre uma base que vaza.
8. Publicar o servidor com `npx supabase functions deploy mcp --no-verify-jwt`.
9. Segunda prova, `npm run test:mcp` : 20 casos.
10. Abrir a tela com `npm run ui` : as duas clínicas lado a lado e a tentativa de acesso cruzado barrada pelo banco.

SOBRE A CHAVE SECRETA

A `service_role` ignora a RLS por completo. Ela é usada em exatamente dois lugares: criar os usuários de teste e ler as contagens de controle. Nunca entra no servidor MCP nem na tela, e o `.env` não deve ser comitado.

Comandos

Comando	O que faz
<code>npm run seed</code>	Cria os usuários e repopula os dados
<code>npm run test:rls</code>	23 casos de isolamento no banco
<code>npm run test:mcp</code>	20 casos através do servidor MCP
<code>npm run ui</code>	A tela, em <code>http://127.0.0.1:4321</code>
<code>npm run demo</code>	A mesma pergunta como os dois médicos, no terminal
<code>npm run mcp:config</code>	Gera o <code>.mcp.json</code> para o Claude Code, um servidor por médico
<code>npm run token -- medicoAlfa</code>	Imprime um token para curl ou MCP Inspector
<code>npm run verify:local</code>	Roda as policies em PostgreSQL puro, sem Docker

Quando algo não funciona

Sintoma	Causa provável
<code>falta a variavel SUPABASE_URL</code>	O <code>.env</code> não existe ou tem o valor de exemplo
Teste de RLS falha logo no caso 1	Schema não aplicado, ou aplicado pela metade
<code>test:mcp</code> dá 401 em tudo	JWT Keys do projeto ainda simétricas (passo 3)
Tela diz "Falta apontar para o seu projeto"	Falta criar o <code>web/config.js</code>
Tela carrega sem pacientes	O seed não rodou, ou rodou em outro projeto
<code>Load failed</code> no navegador	URL errada no <code>config.js</code> ou projeto pausado

7 Aplicabilidade e próximos passos

O padrão vale para qualquer produto em que organizações distintas compartilham um banco e alguém, pessoa ou agente, consulta esses dados em nome de um usuário.

O que se aproveita

- A cadeia de pertencimento e o raciocínio de cada policy
- A separação no banco, não no código da aplicação
- O servidor MCP com ferramentas de domínio, sem acesso livre
- A estratégia de teste com controle por `service_role`

O que não se aproveita

- O schema em si: quatro tabelas escolhidas para isolar o problema, não para representar um domínio real
- Num sistema de verdade, as policies crescem com as tabelas, e a consistência entre elas passa a custar

GOVERNANÇA

Não existe credencial fixa no servidor, nenhuma chave que ignora a RLS no caminho da consulta, e a autorização vem do token, não de um valor enviado pelo cliente. É o oposto do padrão comum de embutir URL e chave no código da aplicação.

Próximos passos

#	Ponto	Impacto hoje	Melhoria sugerida
1	CI incompleto	A bateria do banco roda a cada alteração; a do MCP e o typecheck da função são manuais. Uma regressão passa despercebida.	Primeira coisa a fazer, e barata: rodar <code>deno task check</code> e <code>test:mcp</code> no CI.
2	OAuth para clientes externos	O servidor só é utilizável por um agente embutido no próprio produto.	Avaliar o OAuth Consent block da Supabase; as policies vão precisar dizer o que vale para cada <code>client_id</code> .
3	Schema mínimo por escolha	Um produto real tem dezenas de tabelas, e o custo de manter policies consistentes cresce.	Replicar o padrão em vez de inventar por tabela: mesma cadeia, mesmos helpers, garantia estrutural onde houver denormalização.
4	Sem validação com dado real	Comportamento sob volume, com índices reais e concorrência, não foi medido.	Rodar a bateria contra volume representativo e observar o plano de execução com RLS ativa.

Organização do código

Caminho	Conteúdo
<code>supabase/migrations/</code>	Schema, helpers de RLS e policies, em três arquivos por intenção
<code>supabase/functions/mcp/</code>	O servidor MCP, com as ferramentas em <code>tools/</code>
<code>scripts/</code>	Seed, as duas baterias de teste e utilitários de demonstração
<code>db/local-verify/</code>	Harness que roda as policies em PostgreSQL puro
<code>web/</code>	A tela de demonstração, só arquivos estáticos